

06

BOOK 06 / 08

중요한 정보 보호

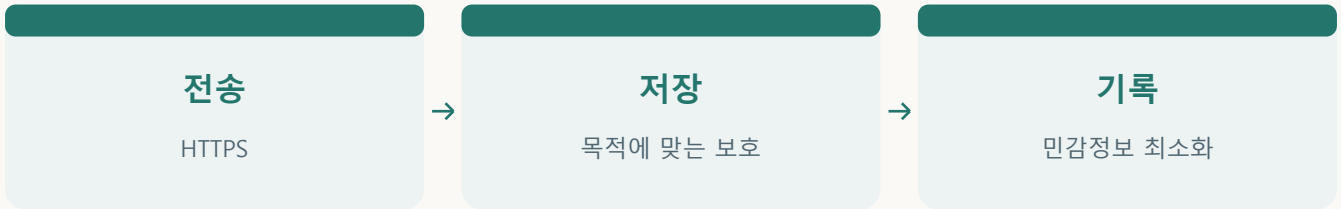
저장·전송·로그에 무엇이 남는가?

개념 이해 / 사례 읽기 / 확인 문제 약 15분

비공개 학습 노트 서비스를 따라 배우는 보안의 기본

이 책을 읽는 방법

정보가 남는 모든 위치를 보호 대상으로 봅니다.



읽기 순서

학습 목표와 선수 지식	3
시작 상황: DB를 보호했는데 로그에 남았다면?	3
개념 설명	3
정보가 남는 위치를 따라가기	4
개발·검수 시 확인사항	4
핵심 요약	5
확인 문제	6
정답과 해설	7
다음 학습과 출처	7

사례를 먼저 읽고, 개념과 코드의 차이를 확인한 뒤 문제를 풀어 보세요. 정답과 해설은 문제 다음 페이지에서 시작합니다.

공통 사례 / 비공개 학습 노트

민서(사용자 101)의 노트는 10번, 준호(202)의 노트는 20번입니다. 모든 노트는 비공개이며, 로그인한 소유자만 조회·수정할 수 있습니다. 제목은 1~80자의 일반 텍스트입니다.

비밀번호·세션 토큰은 로그에 남기지 않고, 권한을 확인할 수 없으면 본문을 보내지 않습니다. 이 가상 서비스의 예제는 학습용 축약본이며 실제 서비스의 계정·데이터·구현과 무관합니다.

학습 목표와 선수 지식

입력·저장·출력의 차이를 이해했다고 가정합니다. 학습 후에는 비밀번호·개인정보·API 키의 처리 목적을 구분하고 암호화·해시·인코딩을 혼동하지 않을 수 있습니다.

시작 상황: DB를 보호했는데 로그에 남았다면?

개발자가 로그인 오류를 찾으려고 요청 본문 전체를 로그에 남겼습니다. 요청에는 비밀번호도 포함되어 있습니다. 계정 DB의 비밀번호 저장 방식을 개선했더라도 로그에 원문이 남는다면 다른 노출 경로가 생깁니다.

개념 설명

방법	핵심 성격	주의할 점
암호화	키를 가진 주체가 원래 정보를 복원하도록 보호	키가 함께 노출되거나 권한이 넓으면 보호 효과가 약해짐
해시	입력으로부터 고정 길이 값을 계산, 복호화 기능은 없음	추측 가능한 입력은 후보를 대입해 맞힐 수 있음
인코딩	다른 형식에서 표현·전송하기 위해 값을 변환	Base64 같은 변환은 비밀정보 보호 수단이 아님

비밀번호는 일반적으로 원문을 다시 보여 줄 필요가 없습니다. 입력한 비밀번호가 맞는지를 확인하면 됩니다. 따라서 비밀번호 저장용으로 설계된 해시 방식과 사용자별 솔트, 적절한 비용 설정을 제공하는 검증된 라이브러리를 사용합니다. 솔트는 같은 비밀번호가 같은 저장값으로 나타나는 것을 줄이는 값이며 비밀키와는 다릅니다. 일반 해시 함수를 한 번 호출하는 것만으로 비밀번호 저장을 설계하지 않습니다.

API 키는 외부 서비스 호출 때 원래 값이 필요한 경우가 많습니다. 이때는 비밀번호처럼 해시해서 끝낼 수 없습니다. 접근 가능한 주체, 비밀 저장 위치, 교체와 폐기 방법을 함께 정해야 합니다.

정보가 남는 위치를 따라가기

정보	필요한 사용	피해야 할 처리
로그인 비밀번호	인증 과정에서 검증	로그·분석 이벤트·오류 응답에 원문 포함
세션 토큰	요청의 로그인 상태 확인	주소나 일반 로그에 기록
비공개 노트	소유자가 조회·수정	디버깅 편의를 위해 본문 전체를 기록
외부 API 키	서버가 외부 서비스 호출	브라우저로 전달되는 코드에 서버 비밀키 포함

다음은 로그 정책을 설명하는 의사코드입니다.

PSEUDOCODE / 흐름 읽기

문제가 있는 기록:

요청 본문 전체와 세션 쿠키를 그대로 로그에 넣는다

개선한 기록:

사건 종류 = "로그인 실패"

결과 = "자격 증명 불일치"

요청 추적 ID = 서버가 발급한 식별자

비밀번호·세션 토큰·노트 본문은 넣지 않는다

추적 ID는 요청을 연결해 조사하기 위한 값이며 로그인 자격 증명이 아닙니다. 필요에 따라 기록하는 사용자 식별자나 접속 정보도 민감할 수 있으므로 접근 권한과 보관 기간을 정해야 합니다.

개발·검수 시 확인사항

HTTPS는 전송 구간을 보호하지만 서버의 로그나 잘못된 접근 권한까지 해결하지 않습니다. 저장 암호화 역시 애플리케이션이 권한 없이 복호화한 내용을 내보내는 문제를 막아 주지 않습니다. 입력부터 저장·백업·로그·응답까지 실제로 남는 복사본을 살펴야 합니다.

비밀정보를 발견했다면 문자열을 소스에서 지우는 것만으로 대응이 끝나는 것은 아닙니다. 노출 범위를 확인하고 해당 비밀의 폐기·교체 필요성을 판단해야 합니다.

핵심 요약

- 암호화·해시·인코딩은 서로 다른 목적을 가집니다.
- 비밀번호 검증과 복원해야 하는 비밀의 보관은 요구가 다릅니다.
- 로그와 오류 응답도 정보가 유출될 수 있는 출력 경로입니다.

확인 문제

1. 비밀번호를 Base64로 바꾸어 저장하면 원문이 안 보이므로 안전할까요?

2. HTTPS를 사용한다면 요청 본문 전체를 로그에 남겨도 될까요?

3. 서버의 외부 API 키를 브라우저 코드에 넣고 문자열을 복잡하게 섞으면 비밀로 유지할 수 있을까요?

정답과 해설

1. 아닙니다. Base64는 공개된 방법으로 원래 값을 복원할 수 있는 인코딩입니다.
2. 아닙니다. 전송 보호와 서버 내부 기록은 별개입니다. 불필요한 민감정보 기록을 줄여야 합니다.
3. 아닙니다. 브라우저에 전달된 값은 이용자가 확인할 수 있다고 가정해야 합니다. 서버용 비밀의 사용 위치를 바꿔야 합니다.

다음 학습과 출처

비밀번호 전용 해시와 솔트: [OWASP Password Storage](#). 로그의 민감정보 제한: [OWASP Logging](#). 다음에는 오류가 발생했을 때 유지해야 할 보호 조건을 살펴봅니다.

나의 한 문장

이 강의에서 배운 내용을 동료에게 설명한다면 어떻게 말할까요?
